

Build Your Second Brain

Turn your notes into a folder your AI can think with — and learn to *drive* the agent while you do it. No coding needed.

Day 1 got you a live site. Day 2 goes deeper: you'll learn how the agent actually "sees" your work (tokens & context), how to brief, plan, and debug it like a teammate, and you'll build a **second brain** — the memory your Day-3 chat agent will answer from.

What's inside

1. **Tokens & context** — how the agent reads, and why it matters
2. **Drive the agent** — brief, plan, read the diff, debug
3. **Start your own project** — your folder, not the course folder
4. **Build your second brain** — capture notes into a vault
5. **Ask your brain** — grounded answers, only from your notes
6. **See it (optional)** — your vault in Obsidian
7. **Make it last** — models & usage limits
8. **Prompt reference** — every prompt in one place
9. **You did it** — the checklist

What you'll need

- The setup from Day 1 (the **Claude app**, Node, Git, a GitHub account). No new installs — see [prep/](#) if you're missing anything.
- **Some raw material** — your CV, a bio, project notes, or a few sentences about yourself. (Product notes work too if you're building for customers.)
- Your paid **Claude plan** (Pro, from \$20/month).

1. Tokens & context

Before you build, a bit of literacy that makes everything else click.

What's a token?

A **token** is a word or a word-piece — the chunks the model reads. It's the unit the model **reads**, *and* the unit your plan is **billed and limited** in.

See it for yourself: paste a paragraph into the OpenAI tokenizer at platform.openai.com/tokenizer and watch it split into tokens. Try a bit of your CV.

The context window

The **context window** is everything the agent can see *right now*, measured in tokens — and it's **finite**. Long, messy chats fill it up, and the agent gets **vaguer and slower**. More isn't better; a focused window is a sharp agent.

Managing state with `/clear`

Starting a new, unrelated task? Run `/clear` to wipe the chat and start fresh.

- **Persists:** your **files** and `CLAUDE.md`.
- **Doesn't persist:** the **chat history**.

```
Summarise what we've done in 3 bullets so I can keep it, then  
I'll /clear and start fresh on the next task.
```

Rule of thumb: one task = one focused session. Your *files* are the memory — not the chat.

2. Drive the agent

Today you stop "just prompting" and start **directing**. Four moves.

Brief like a teammate

- Be **specific** about what you want.
- Give the **context** it needs.
- Say what "**done**" looks like.

Plan first

Ask for a plan *before* it builds. Read it. Steer or interrupt if it's heading the wrong way — it's cheaper to fix a plan than to unpick a wrong build.

```
Before you change anything, give me a short plan for how you'll  
do this, and wait for my OK.
```

Read the diff, Accept deliberately

When Claude proposes changes, **read the diff** and **Accept** on purpose — never blindly. You approve every step; that's the job.

Debug calmly

When something breaks, work it — don't panic:

1. **Read** the actual error.
2. **Give it back** to the agent.
3. **Narrow it down** together.
4. Still lost? **Revert** to your last good commit.

```
This is the exact error: [paste]. Don't guess – find the cause,  
explain it to me simply, then fix it.
```

3. Start your own project

Same golden rule as Day 1:

The course repo is a textbook, not your workbook. Build in **your own folder** that you own. (More: [prep/working-on-your-own-projects.md](#).)

In the **Code** tab, paste:

```
Make a new folder called "my-second-brain" in my home directory – NOT inside the jungli course folder. Copy the second-brain skeleton (including the vault/ folder) into it, start a fresh git repo, and confirm we're working in my-second-brain.
```

Full build flow lives in the starter's [PROMPTS.md](#).

4. Build your second brain

A **second brain** is a folder of notes the agent can read and answer from. The trick is *how* the notes are shaped.

Intake — what is this brain about?

- Whose / what knowledge is this? (you, for a job hunt — or your product, for customers)
- What questions should it eventually answer?
- What source material do you have? (CV, notes, a website, product docs)

Capture — one messy document → tidy notes

Point Claude at your material:

```
Read [paste text, or point to a file]. Break it into atomic markdown notes in vault/, one idea per file, with clear titles. Add [[wikilinks]] between related notes. Then create vault/index.md listing and grouping the notes.
```

You get **atomic notes** (one idea each), `[[links]]` between them, and an **index** — exactly like the sample `finished/` vault (Sam Rivera).

Organise — tighten it

Review vault/. Merge duplicate notes, fix broken links, and group the notes under clear headings in index.md. Keep each note to one idea.

Why atomic + linked? A tight, well-organised vault gives *better answers* than one big messy note. Keep raw dumps (the whole CV) **out** of vault/ — the vault holds the distilled notes.

5. Ask your brain

The payoff. Ask it real questions:

Using ONLY the notes in vault/, answer this the way I'd want a recruiter to hear it: [their likely question]. If the answer isn't in the vault, tell me what's missing instead of guessing.

Ask it **three real questions** and watch what it does when it doesn't know.

The grounding trap — and the point of this build: the agent answers **only** from your vault. If something isn't there, it should **say so**, not invent it. Grounded answers you can trust beat confident guesses. This is the memory your **Day-3 chat agent** will run on.

6. See it (optional) — Obsidian

Your vault is just markdown files, so they open beautifully in [Obsidian](#) (a free notes app) — with a **graph view** that draws every note and the links between them. Same files, two windows.

Keep Obsidian open on your vault while you prompt Claude, and you'll *watch* your second brain being built in real time. Full steps: [OBSIDIAN.md](#) .

Optional. Skip it and your brain still works perfectly — it's the "see it" bonus, and a great way to spend extra time once the core build is done.

7. Make it last — models & limits

You'll build all day, so spend your usage wisely.

Right model for the job

- **Lighter** (Haiku / Sonnet) — fast, great for simple or repetitive work.
- **Heavier** (Opus) — for genuinely hard problems.
- Using the big model for *everything* burns your limit faster.

This is a simple, repetitive edit. Which model should I use to save my usage, and why?

Limits on Pro

- Usage is **shared** with your normal Claude chats — one pool.
 - Two limits: a **5-hour session window** (resets every 5 hours) *plus* a **weekly cap**. Anthropic doesn't publish exact numbers; check yours in **Settings** → **Usage** or with `/status` . (More: [prep/claude-plans.md](#) .)
 - Make it last: right model · `/clear` per task · don't leave it idle · lean on **files**, not one endless chat.
-

8. Prompt reference

Every prompt from this guide, in one place. Copy and customise.

Keep a summary before you `/clear`

```
Summarise what we've done in 3 bullets so I can keep it, then  
I'll /clear and start fresh on the next task.
```

Plan first

```
Before you change anything, give me a short plan for how you'll  
do this, and wait for my OK.
```

Debug

```
This is the exact error: [paste]. Don't guess – find the cause,  
explain it to me simply, then fix it.
```

Start your own project

```
Make a new folder called "my-second-brain" in my home directory –  
NOT inside the jungli course folder. Copy the second-brain  
skeleton (including vault/) into it, start a fresh git repo, and  
confirm we're working in my-second-brain.
```

Capture notes

```
Read [paste text, or point to a file]. Break it into atomic  
markdown notes in vault/, one idea per file, with clear titles.  
Add [[wikilinks]] between related notes. Then create  
vault/index.md listing and grouping the notes.
```

Organise

Review vault/. Merge duplicate notes, fix broken links, and group the notes under clear headings in index.md. Keep each note to one idea.

Ask your brain

Using ONLY the notes in vault/, answer this the way I'd want a recruiter to hear it: [their likely question]. If the answer isn't in the vault, tell me what's missing instead of guessing.

Pick a model

This is a simple, repetitive edit. Which model should I use to save my usage, and why?

9. You did it

If you followed every step, you now have:

- ☐ A feel for **tokens, context, and /clear**
- ☐ The four driving moves — **brief, plan, read the diff, debug**
- ☐ Your own **second brain**: atomic, linked notes with an index
- ☐ Answers **grounded only in your notes**
- ☐ Habits that make your **Pro plan last** a full build day

Your second brain is the **context** for Build 3 — a chat agent that answers as *you* (or your product). Same files, bigger payoff.

Built for the Jungli AI Learning Residency · AI & Agents track.